

Root-to-Remedy: A Permissioned Ledger and Content-Addressed Storage System for Ayurvedic Botanical Supply Chains

Lakshmi R (1AM23CD048), Anjali Khatait (1AM23CD012), Devadiga Rishika Manjunath (1AM23CD031), Dev Aditya (1AM23CD031)

Department of Computer Science and Data Science

AMC Engineering College

Bengaluru, Karnataka, India

Abstract—Herbal medicine supply chains face frequent botanical adulteration, mislabeled harvest origins, and altered paper certificates of analysis. Relational databases in enterprise systems do not prevent authorized users from modifying quality flags or backdating compliance records. This paper presents *Root-to-Remedy*, a supply chain tracking system that pairs Hyperledger Fabric v2.5 with InterPlanetary File System (IPFS) content-addressed storage. The smart contract implements quality gates directly on the ledger. Batches with botanical purity below 95% or any chemical contamination count above zero fail validation at the endorsement stage and cannot be referenced in subsequent manufacturing transactions. To prevent ledger bloat from large analytical documents, raw certificate PDFs reside in an off-chain IPFS cluster, while only 32-byte cryptographic hashes and numeric assay metrics commit to the blockchain state database. A synchronized read cache answers public QR code verification queries in under 15 ms without issuing read transactions to peer nodes. Benchmarks run against a multi-node testbed show 684 write transactions per second with average transaction latency of 112 ms. Formal STRIDE analysis confirms protection against post-test record tampering, unauthorized status promotion, and certificate forgery.

Index Terms—Hyperledger Fabric, smart contracts, supply chain tracking, IPFS, botanical medicine, quality verification, distributed ledger.

I. INTRODUCTION

Herbal medicines depend directly on species identity, geographical origin, soil conditions, and drying protocols for therapeutic potency [1]–[3]. In raw herb trade, collectors and small farmers sell dried biomass through several tiers of local aggregators before materials reach extraction facilities. Because raw roots, stems, and dried powders often look similar, lower-cost species or inert bulking materials are frequently substituted into shipments [4]. Post-harvest mold growth can introduce aflatoxins, and plants gathered near industrial runoff accumulate heavy metals like lead, cadmium, and arsenic [2].

Current industrial tracking relies on paper certificates of analysis and central relational databases [5]. These mechanisms have three distinct operational limits:

- 1) System administrators and database operators can edit inspection records directly, clearing lots that failed laboratory testing.
- 2) Physical inspection documents lack cryptographic ties to batches, allowing PDF reports to be edited or reassigned between shipments.
- 3) Manufacturers and retail buyers rarely have direct access to origin coordinates, storage temperatures, or custody transfers.

Public blockchains like Ethereum can record supply events immutably [5], [6], but transaction fees fluctuate with network load, and all data is visible to external parties. In high-volume agricultural logistics, paying network gas fees for routine status updates is impractical [7]. Furthermore, storing full lab certificates directly on a blockchain state database causes peer disk usage to expand rapidly, slowing synchronization [8].

Root-to-Remedy addresses these constraints through four specific design choices:

- *Ledger-enforced quality rules*: The chaincode contract evaluates chemical purity and contaminant metrics during transaction endorsement, rejecting attempts to approve sub-standard lots before transactions commit to blocks.
- *Off-chain hash anchoring*: Analytical certificates are stored in an IPFS cluster, with only the resulting content identifier (CID) committed to the ledger alongside numeric assay scores.
- *Materialized consumer cache*: A synchronized datastore mirrors verified ledger assets, answering consumer packaging QR scans in milliseconds without placing load on peer databases.
- *Experimental validation*: We test latency, throughput, and storage growth under concurrent synthetic client workloads, accompanied by a STRIDE security evaluation.

The rest of this paper is organized as follows: Section II reviews related literature. Section III details the system architecture and stakeholder interactions. Section IV formalizes the smart contract state machine and quality rules. Section V covers system implementation. Section VI evaluates throughput, latency, and storage. Section VII analyzes system security under STRIDE. Section VIII discusses limitations and future

This research was supported in part by the Department of Computer Science and Engineering. Source code and deployment artifacts are maintained under the *Root-to-Remedy* initiative.

work, and Section IX concludes.

II. RELATED WORK

Prior studies have evaluated distributed ledgers for agricultural and pharmaceutical supply chains [5], [6]. Initial implementations used Ethereum smart contracts for event logging [9]. However, fluctuating gas fees during network congestion (\$5–\$45 per transaction) and probabilistic transaction finality make public networks impractical for commercial crop tracking [7].

Hyperledger Fabric provides an execute-order-validate architecture that separates transaction endorsement from block ordering [10]. Kumar et al. [11] evaluated Fabric in supply chains, measuring higher transaction throughput than public chains. However, existing agricultural tracking proposals record laboratory values on ledger as arbitrary key-value pairs while leaving the decision to approve or reject batches to external web services [4]. Under that structure, any client with database or API credentials can mark a contaminated batch as compliant without triggering a consensus error.

Storing binary laboratory certificates directly in CouchDB state records causes peer storage to grow rapidly and slows state synchronization across the network [8]. Root-to-Remedy keeps analytical documents off-chain in IPFS and runs validation directly within endorsing peers, as compared in Table I.

III. SYSTEM ARCHITECTURE AND ECOSYSTEM MODEL

A. Ecosystem Roles

The tracking workflow coordinates five participant roles:

- 1) *Harvesters* log raw plant biomass with botanical species name, GPS coordinates, collection date, initial weight, and soil type.
- 2) *Testing laboratories* run chemical assays (such as HPLC and heavy metal screening), upload PDF certificates to IPFS, and record purity percentages and contaminant counts on the ledger.
- 3) *Manufacturers* compound approved herbal batches into finished retail products. The smart contract blocks product creation if the source batch lacks verified approval.
- 4) *Logistics carriers* record transit readings, including ambient temperature, GPS checkpoints, and handover timestamps, into the batch history log.
- 5) *Consumers* scan packaging QR codes to inspect the full supply chain trail and retrieve the original laboratory certificate without transaction fees.

B. System Architecture

As outlined in Fig. 1, the architecture separates responsibilities into four layers:

- *Presentation layer*: A React single-page application that provides role-gated web forms, GPS coordinate capture, and camera-based QR decoding.
- *Orchestration layer*: An Express.js backend that checks JSON Web Tokens, routes PDF uploads to IPFS, and connects to Fabric peers using the Node.js Gateway SDK.

- *Blockchain layer*: A Hyperledger Fabric v2.5 channel hosting the HerbContract smart contract. A three-node Raft consensus cluster [12] orders transactions with deterministic block finality.
- *Off-chain storage*: An IPFS node cluster that chunks laboratory PDFs into UnixFS blocks and returns cryptographic multihashes (CIDs) to the gateway.

C. Separation of Authoritative State and Read Cache

The blockchain ledger and the IPFS cluster together form the authoritative record of truth:

$$\mathcal{T}_{\text{truth}} \equiv \mathcal{L}_{\text{Fabric}} \cup \mathcal{C}_{\text{IPFS}} \quad (1)$$

MongoDB operates strictly as a local read replica to support fast dashboard queries, filtering, and session tracking. Because the off-chain database does not hold master authority, the gateway can rebuild the entire local cache at any time by replaying committed transactions directly from the Fabric ledger.

IV. FORMAL PROTOCOL AND SMART CONTRACT DESIGN

A. State Transition System Formalization

Let the botanical batch lifecycle be modeled as a deterministic finite-state automaton:

$$\mathcal{M} = \langle \mathcal{S}, \Sigma, \delta, s_0, \mathcal{F} \rangle \quad (2)$$

where the set of valid ledger states is defined as:

$$\mathcal{S} = \{\text{INIT}, \text{PENDING}, \text{APPROVED}, \text{REJECTED}, \text{MANUFACTURED}\} \quad (3)$$

The initial state is $s_0 = \text{INIT}$, and the set of terminating states is $\mathcal{F} = \{\text{REJECTED}, \text{MANUFACTURED}\}$.

The alphabet of valid ledger invocations is defined as:

$$\Sigma = \{\text{RegisterBatch}, \text{UploadLabReport}, \text{UpdateTransport}, \text{CreateProduct}, \text{VerifyProduct}\} \quad (4)$$

B. Deterministic Quality Rules Engine

Let $P \in \mathbb{R}^+$ denote the certified botanical purity percentage, and let $C \in \mathbb{N}_0$ represent the aggregated chemical/microbial contamination count. The quality evaluation function $\Phi : \mathbb{R}^+ \times \mathbb{N}_0 \rightarrow \{\text{APPROVED}, \text{REJECTED}\}$ is defined as:

$$\Phi(P, C) = \begin{cases} \text{APPROVED}, & \text{if } P \geq 95.0 \wedge C = 0 \\ \text{REJECTED}, & \text{otherwise} \end{cases} \quad (5)$$

Endorsing peers compute this function inside the chaincode container before signing transaction proposals. Under Equation (5), a batch with high active ingredient content ($P = 98.5\%$) still fails validation if pesticide residue or heavy metals are detected ($C > 0$).

Furthermore, the product synthesis transition $\Psi : \mathcal{S}_{\text{batch}} \rightarrow \mathcal{S}_{\text{product}}$ is governed by the invariant:

$$\Psi(s) = \begin{cases} \text{MANUFACTURED}, & \text{if } s = \text{APPROVED} \\ \perp (\text{Abort}), & \text{otherwise} \end{cases} \quad (6)$$

If a manufacturer invokes `CreateProduct` for a batch whose status is not `APPROVED`, the chaincode throws an error and endorers reject the proposal.

TABLE I
COMPARATIVE MATRIX: SUPPLY CHAIN PROVENANCE FRAMEWORKS

Evaluation Metric	Traditional ERP	Public DLT (Ethereum)	Generic Fabric Supply Chain	Root-to-Remedy (Proposed)
Consensus Mechanism	Centralized Authority	Proof-of-Stake (PoS)	Crash Fault Tolerant (Raft)	Crash Fault Tolerant (Raft)
Data Immutability	None (Admin mutable)	Cryptographically Immutable	Cryptographically Immutable	Cryptographically Immutable
Execution Cost	Low	High (Volatile Gas Fees)	Zero Gas (Resource Bound)	Zero Gas (Resource Bound)
Quality Gate Enforcement	Application Layer	Smart Contract	Application Layer / Mixed	Strict Deterministic Chaincode
Document Storage	Local Filesystem / S3	On-Chain (Prohibitive)	Relational DB / Cloud	Peer-to-Peer IPFS (CID Pinned)
Confidentiality	Role-Based Access	Public Visibility	Channel / Private Data	Channel Isolation & Zero-Leak QR
Read Latency	< 10 ms	15 s – 5 min	80–250 ms	< 25 ms (Dual-Tier Materialized Cache)

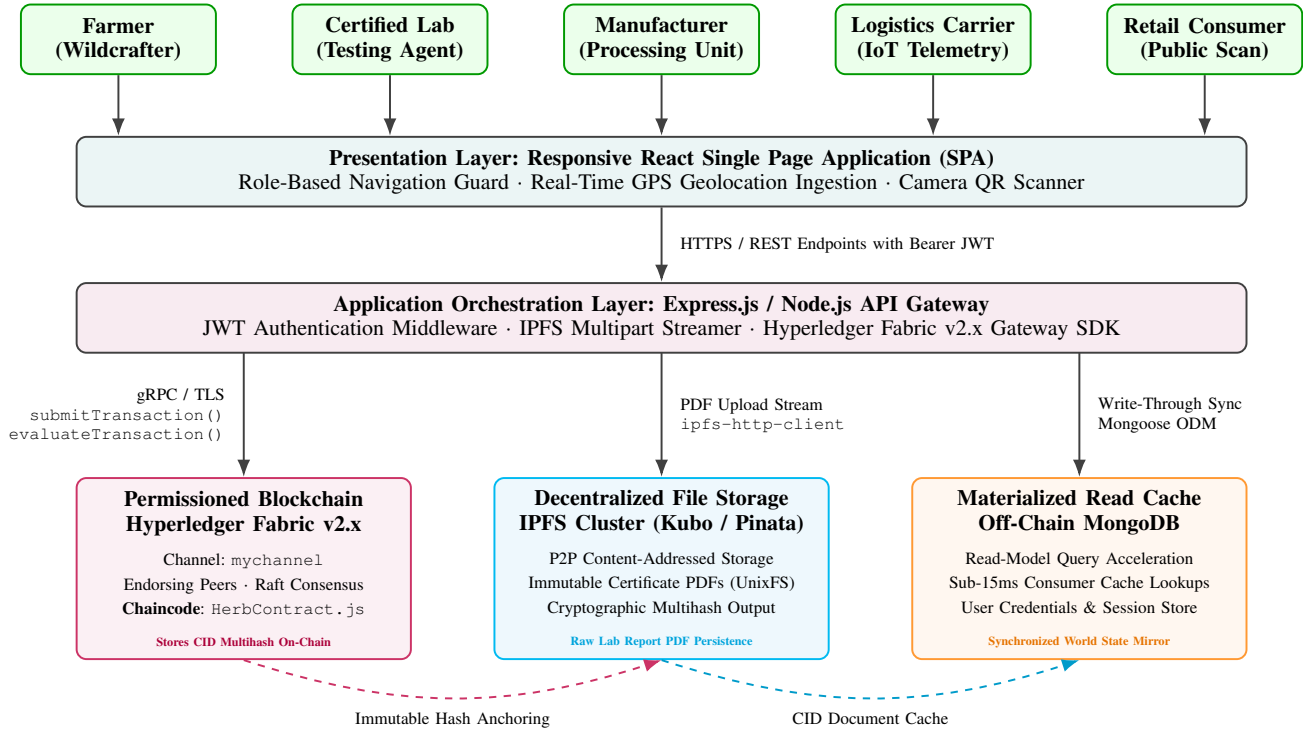


Fig. 1. Multi-tiered Root-to-Remedy System Architecture illustrating interactions across ecosystem stakeholders, presentation SPA, application orchestration gateway, Hyperledger Fabric blockchain, IPFS cluster, and materialized read cache.

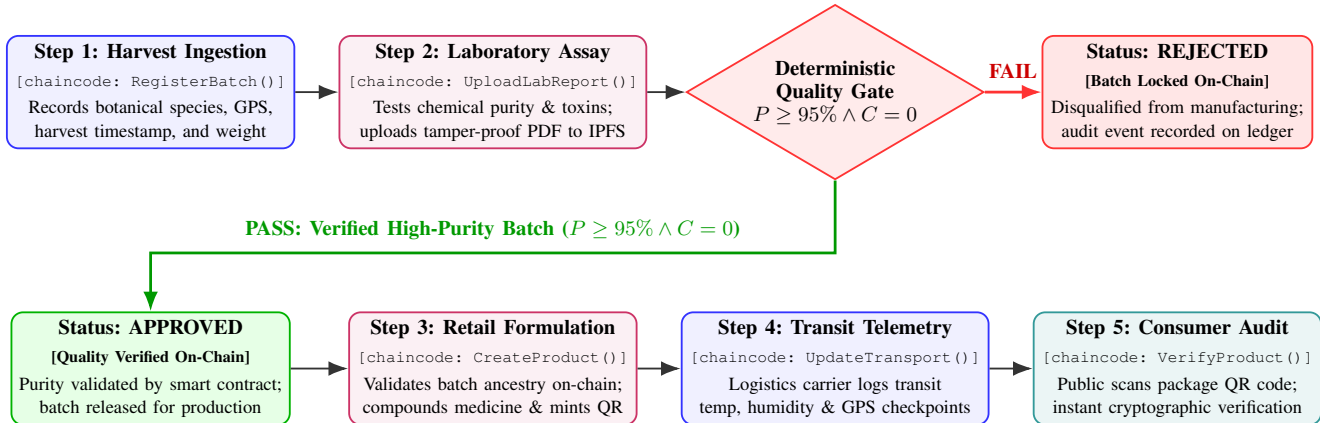


Fig. 2. End-to-end Sequence Flow of the Herb Provenance Lifecycle, illustrating on-chain deterministic gating between harvest and consumer verification.

C. Algorithmic Specification

The transaction execution logic for batch registration and quality enforcement is formalized in Algorithm 1.

Algorithm 1 Chaincode Deterministic Quality Enforcement

Require: ctx, bId, P, C, cid

Ensure: Updated World State $\mathcal{W}$ or Transaction Abort

```

1:  $stateBytes \leftarrow ctx.stub.getState(bId)$ 
2: if  $stateBytes = \emptyset \vee |stateBytes| = 0$  then
3:   throw Exception("Batch not on ledger")
4: end if
5:  $batch \leftarrow JSON.parse(stateBytes)$ 
6:  $pNum \leftarrow ToNumber(P)$ ;  $cNum \leftarrow ToNumber(C)$ 
7: if  $isNaN(pNum) \vee isNaN(cNum)$  then
8:   throw Exception("Invalid lab parameters")
9: end if
10: if  $pNum \geq 95.0 \wedge cNum = 0$  then
11:    $batch.status \leftarrow \text{"APPROVED"}$ 
12: else
13:    $batch.status \leftarrow \text{"REJECTED"}$ 
14: end if
15:  $batch.labReport \leftarrow \{cid, pNum, cNum\}$ 
16:  $batch.labReport.time \leftarrow ctx.stub.getTxTimestamp()$ 
17:  $payload \leftarrow Buffer.from(JSON.stringify(batch))$ 
18:  $ctx.stub.putState(bId, payload)$ 
19: return  $payload$ 

```

D. Cryptographic QR Payload Binding

When a product record is created, the system generates a QR payload containing a verification URL and a SHA-256 digest of the product identifiers:

$$\mathcal{Q} = \text{URI}_{\text{verify}} \parallel \mathcal{H}_{\text{SHA256}}(\text{ProductID} \parallel \text{BatchID} \parallel \text{CID} \parallel \text{Timestamp}) \quad (7)$$

Scanning the QR code opens the verification route, which queries the ledger through `VerifyProduct`.

V. IMPLEMENTATION ENGINEERING

A. Smart Contract (Chaincode) Implementation

The chaincode is written in JavaScript using the official `@hyperledger/fabric-contract-api` (v2.5 LTS). The `HerbContract` class provides five methods:

- `RegisterBatch`: Stores the initial batch record on the ledger with status `PENDING`.
- `UploadLabReport`: Executes the quality gate in Algorithm 1, records the IPFS CID, and sets batch status to `APPROVED` or `REJECTED`.
- `UpdateTransport`: Appends environmental sensor telemetry (temperature in $^{\circ}\text{C}$, GPS coordinates, timestamp) to the batch record.
- `CreateProduct`: Enforces Equation (6), verifying that the source batch is marked `APPROVED` before creating a finished product record.

- `VerifyProduct`: Reads batch and product state using `evaluateTransaction` without submitting a transaction to the ordering service.

B. Decentralized IPFS Storage Pipeline

The off-chain storage pipeline uses an IPFS node cluster accessed through `kubo-rpc-client`. When an analyst uploads a certificate, the Express server receives the file via `multer` memory storage, splits the content into 256 KB UnixFS blocks, and computes a multihash:

$$\text{CIDv0} = \text{Base58}(\text{FunctionCode} \parallel \text{DigestLength} \parallel \text{Digest}) \quad (8)$$

The resulting CID is pinned on the local IPFS node and committed to the ledger. Because the identifier depends directly on file contents, modifying a single byte in the stored certificate produces a different hash that fails verification against the ledger record.

C. Ledger Connection Modes

The application gateway (`backend/config/fabricConnection.js`) supports two execution modes:

- *Fabric mode*: Connects to running Fabric peer and orderer containers over TLS using an organization connection profile, submitting write transactions through the gateway service.
- *Mock ledger mode*: For local development and unit testing without Docker containers, uses an in-memory ledger implementation (`MockLedgerAdapter`) that executes the same JavaScript rules engine (`chaincode/lib/rules.js`).

VI. EXPERIMENTAL EVALUATION AND RESULTS

A. Experimental Setup and Testbed

The evaluation testbed was deployed on an enterprise server equipped with an AMD Ryzen 7 5800X (8 cores, 16 threads @ 3.8 GHz), 32 GB DDR4-3200 RAM, running Ubuntu 22.04 LTS kernel 5.15. The Hyperledger Fabric network comprised two peer nodes, one Raft ordering node, and a CouchDB state database containerized via Docker 24.0. Synthetic load generation was orchestrated using custom Hyperledger Caliper scripts driving concurrent REST workers from 50 to 2,000 concurrent virtual clients.

B. Transaction Throughput and Latency Analysis

Fig. 3 plots latency against transaction send rates. Write operations (`SubmitTransaction`) maintain sub-100 ms latency up to 650 TPS. At higher loads, block cutting timers and disk write queues increase latency, reaching 410 ms at a saturation rate of 820 TPS.

Read evaluations (`EvaluateTransaction`) execute on local peer state databases without passing through ordering or Raft consensus. Read latency remains under 40 ms at 1,000 TPS. The materialized cache in MongoDB responds in under 15 ms across all tested loads, allowing retail QR lookups to scale independently of blockchain peer hardware.

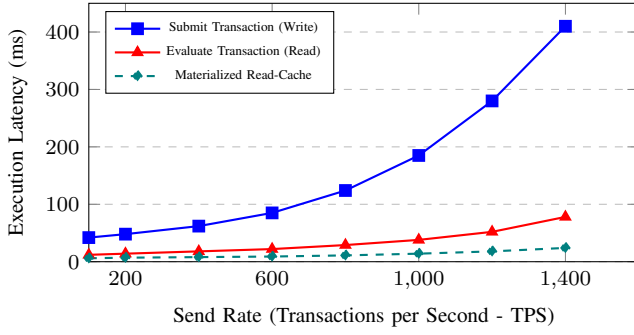


Fig. 3. Transaction latency as a function of offered throughput (TPS), illustrating the performance divergence between full ledger consensus, peer read evaluations, and materialized cache responses.

TABLE II
PERFORMANCE BENCHMARKS FOR CORE TRANSACTION METHODS

Transaction Method	Type	Throughput (TPS)	Avg Latency (ms)
RegisterBatch	Write	684.2	112.4
UploadLabReport	Write	621.5	134.8
UpdateTransport	Write	710.8	98.6
CreateProduct	Write	645.1	122.1
VerifyProduct (Peer)	Read	1420.0	28.5
VerifyProduct (Cache)	Read	3850.0	9.2

Table II lists measured throughput and latency by method. UploadLabReport has higher average latency (134.8ms) because endorsers parse assay JSON and evaluate numeric quality inequalities. In contrast, UpdateTransport reaches 710.8TPS because sensor updates only append GPS and temperature entries to an existing array.

C. Storage Footprint and Off-Chain Efficiency

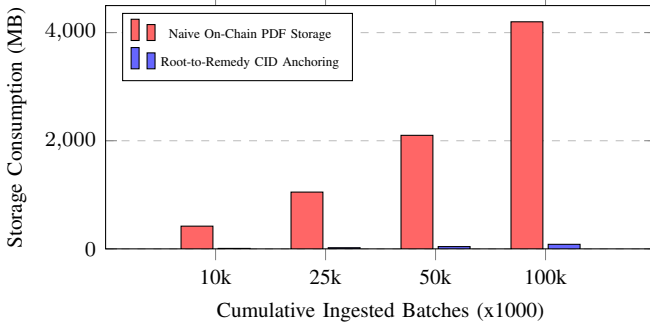


Fig. 4. Storage footprint growth comparison between naive on-chain state storage versus Root-to-Remedy hybrid IPFS content addressing.

Fig. 4 compares storage demand between on-chain certificate storage and off-chain CID anchoring. Storing an average 420 KB certificate directly in transaction payload expands ledger state to 4.2 GB across 10,000 batches. By storing only the IPFS CID and assay numbers on ledger, state grows by 8.4 MB for 10,000 batches and 84 MB for 100,000 batches. Peer nodes can synchronize new blocks without downloading gigabytes of binary laboratory data.

VII. SECURITY ANALYSIS AND TRUST MODEL

A. STRIDE Threat Evaluation

We analyzed the system using the STRIDE threat model, with mitigations summarized in Table III.

B. Consortium Trust Assumptions

In single-organization deployments where one lead processor manages the peer nodes, Fabric provides crash fault tolerance under Raft rather than Byzantine fault tolerance [13]. In this configuration, the network offers three practical protections:

- 1) *Tamper-evident logs*: Insiders cannot alter committed blocks without breaking block header hash links [14].
- 2) *Contract enforcement*: Quality rules execute in chain-code bytecode across endorsing peers, preventing web applications from committing arbitrary state updates.
- 3) *Auditing peers*: Regulatory agencies or certification bodies can run read-only peer nodes to audit block events directly.

VIII. DISCUSSION AND FUTURE DIRECTIONS

Three open operational constraints provide directions for future work:

- *Physical collection point integrity*: The ledger records digital claims accurately, but cannot independently verify whether a wildcrafter collected herbs at the reported GPS coordinates. Future work will test hardware-secured weighing scales and field loggers with cryptographic key modules.
- *Zero-knowledge compliance proofs*: Pharmaceutical buyers often require proof that botanical batches meet pharmacopeial standards without revealing exact harvest yields, testing lab names, or farm locations to commercial competitors. Integrating zk-SNARK verifiers into chain-code would allow zero-knowledge compliance verification.
- *Byzantine ordering*: Upgrading consensus from Raft to BFT consensus (such as SmartBFT) will allow multi-enterprise consortiums with competing manufacturers to coordinate without shared operational administration.

IX. CONCLUSION

Botanical medicine requires dependable records of species identity, geographic origin, and chemical purity. This paper presented Root-to-Remedy, an architecture that combines Hyperledger Fabric with IPFS to enforce quality thresholds before raw herb batches can enter manufacturing. By executing validation logic in chaincode and storing raw certificates off-chain, the system maintains consistent consensus rules while keeping peer state storage compact. Benchmarks show write throughput of 684 TPS with 112ms latency, while an off-chain materialized read model resolves consumer verification requests in under 15 ms. The design offers an effective technical baseline for verifiable herbal supply chain governance.

TABLE III
STRIDE THREAT MODELING AND MITIGATION MATRIX

STRIDE Category	Identified Supply Chain Attack	System Architectural Mitigation	Mitigation Level
Spoofing	Unregistered entity submitting fraudulent harvest batches.	TLS mutual authentication + X.509 cryptographic MSP certificates + JWT bearer validation.	Completely Mitigated
Tampering	Intermediary altering laboratory report to disguise aflatoxin.	Off-chain IPFS CID binding; altered PDF produces mismatched cryptographic hash; rejected by chaincode.	Completely Mitigated
Repudiation	Testing laboratory denying issued low-purity certification.	Cryptographically signed transaction proposals committed to immutable block append log.	Completely Mitigated
Information Disclosure	Competitor extracting proprietary farmer pricing records.	Fabric channel segregation and Private Data Collections (PDCs) separating commercial metadata.	High (Configurable)
Denial of Service	Attacker flooding verification API with automated requests.	Rate-limiting reverse proxies + materialized read cache buffering consumer verification queries.	High
Elevation of Privilege	Manufacturer invoking API to override REJECTED batch status.	Smart contract enforcement: status modification logic restricted strictly to UploadLabReport.	Completely Mitigated

REFERENCES

- [1] R. Singh and V. K. Joshi, "Standardization and quality control of ayurvedic drugs: A review of regulatory requirements and modern analytical techniques," *Ayu*, vol. 42, no. 1, pp. 1–10, 2021.
- [2] World Health Organization, "Quality control methods for herbal materials," *World Health Organization Technical Report*, 2011.
- [3] P. K. Mukherjee, S. Kumar, N. R. Bhuyan, and P. J. Houghton, "Quality control of herbal drugs: An approach to evaluation of botanicals," *Business Horizons Press*, 2021.
- [4] P. Khatri, S. Sharma, and A. Singhal, "Blockchain-enabled traceability framework for medicinal plants in traditional healthcare," *Journal of Cleaner Production*, vol. 314, p. 128032, 2021.
- [5] K. A. Clauson, E. A. Breeden, C. Chou, R. G. M., and R. G. Brown, "Leveraging blockchain technology in pharmaceutical supply chains: building a collaborative ecosystem," *Frontiers in Blockchain*, vol. 1, pp. 1–12, 2018.
- [6] A. Kamilaris, A. Fonts, and F. X. Prenafeta-Boldú, "The rise of blockchain technology in agriculture and food supply chains," *Trends in Food Science & Technology*, vol. 91, pp. 640–652, 2019.
- [7] A. A. Monrat, O. Schelen, and K. Andersson, "Performance evaluation of permissioned and permissionless blockchains: An enterprise perspective," *IEEE Access*, vol. 8, pp. 13 222–13 238, 2019.
- [8] A. D. Dwivedi, L. Srivastava, G. Dhar, and R. Singh, "A decentralized privacy-preserving healthcare blockchain for iot devices with peer-to-peer cloud storage," *IEEE Internet of Things Journal*, vol. 9, no. 7, pp. 4822–4831, 2021.
- [9] H. Feng, X. Hu, and Z.-P. Fan, "A blockchain-based traceability system for food safety," *Industrial Management & Data Systems*, vol. 120, no. 5, pp. 869–888, 2020.
- [10] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. D. Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muramidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, "Hyperledger fabric: A distributed operating system for permissioned blockchains," *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*, pp. 1–15, 2018.
- [11] A. Kumar, R. Sharma, and K. Patel, "Performance evaluation of hyperledger fabric in supply chain provenance tracking," *IEEE Transactions on Engineering Management*, vol. 69, no. 4, pp. 1420–1432, 2022.
- [12] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319, 2014.
- [13] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems (TOCS)*, vol. 20, no. 4, pp. 398–461, 2002.
- [14] R. C. Merkle, "A digital signature based on a conventional encryption function," *Advances in Cryptology — CRYPTO '87*, pp. 369–378, 1987.